IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project — High Level Design

On

Honeypot Network for EdTech Threat Intel

Student Name:
Dilnas
23BCCE104

Table of Content

1. Introduction 3
1.1. Scope of the Document 3
1.2. Intended Audience 3
1.3. System Overview 3
2. System Design 5
2.1. Architecture Overview 5
2.2. Components 5
2.3. Process Flow 5
2.4. Information Flow 7
2.5. API Catalogue 7
3. Data Design 8
3.1. Data Model 9
3.2. Data Access Mechanism 9
3.3. Data Retention Policies 9
3.4. Data Processing (ETL) 9
4. Interfaces 10
5. State and Session Management 10
6. Caching 11
7. Non-Functional Requirements 11
7.1. Security Aspects 11
7.2. Performance Aspects 11
7.3. Limitations 11
8. References 12

1. Introduction

This document presents the High-Level Design (HLD) of the Honeypot Network for EdTech Threat Intel system. The system is designed to listen on multiple education-themed ports, log every connection attempt in real time, send a fake 'Access Denied' banner to the attacker, and present all captured threat data through a web-based monitoring dashboard. It is inspired by the working principle of enterprise honeypot tools such as Cowrie and T-Pot used by real institutional security teams.

The project simulates a Security Operations Center (SOC) workflow — from the moment an attacker attempts a connection to the moment the incident is recorded and displayed on the dashboard — making it suitable for academic study as well as real-world awareness in the domain of cybersecurity for educational institutions.

1.1. Scope of the Document

This HLD document covers the following aspects of the system:

- Overall system architecture and component interactions
- Application design including process flow and information flow
- Data design including log storage format, access mechanism and retention policies
- Non-functional requirements such as security and performance
- Interface design for the Flask-based dashboard

This document does not cover low-level implementation details such as line-by-line code logic, which are addressed in the Low-Level Design (LLD) document.

1.2. Intended Audience

This document is intended for the following stakeholders:

- Academic evaluators and faculty reviewing the project submission
- Developers or students who may extend or build upon this system
- Cybersecurity enthusiasts seeking to understand honeypot-based threat intel
- Anyone interested in the application of Python sockets in educational network security

1.3. System Overview

The Honeypot Network for EdTech Threat Intel is a real-time cybersecurity monitoring system built using Python, the socket library, threading and Flask. It listens on multiple education-themed ports, captures every connection attempt with IP and timestamp, stores the data in a JSON log, responds with a fake student portal banner, and presents all threat data through an interactive web-based dashboard.

The system mimics the workflow of a real SOC honeypot environment:

- Listen: Bind to ports 8080, 21, 443 and 3306 simultaneously using threading
- Detect: Accept any incoming TCP connection as a potential attack attempt
- Log: Record attacker IP address, port, service name and timestamp
- Respond: Send a fake 'EdTech Student Portal — ACCESS DENIED' banner to the attacker
- Visualize: Display real-time attack statistics, charts and live log table

The system is built using the following technology stack:

Component	Technology	Purpose
Port Listener	Python Socket Library	Binds EdTech ports, accepts connections
Concurrency	Python Threading	Handles multiple ports and connections
Logging	JSON + File I/O	Stores IP, port, service, timestamp records
Fake Response	Python Bytes Encoding	Sends fake Access Denied banner
Dashboard	Flask + HTML + CSS	Real-time web monitoring interface
Visualization	Chart.js	Line chart, doughnut chart, data tables

2. System Design

2.1. Architecture Overview

The system is built as two cooperating Python applications running side by side. The core honeypot (honeypot.py) is a multi-threaded socket server that binds to four EdTech ports and listens for connections indefinitely. The dashboard (dashboard.py) is a separate Flask web server that reads the shared JSON log file and exposes both an HTML monitoring interface and JSON API endpoints. Both applications share state only through the log file, keeping them fully decoupled.

2.2. Components

The process flow of the system follows a sequential pipeline:

- Step 1 – Initialization: Port configuration loaded, log file created/verified on startup
- Step 2 – Binding: Sockets bound to ports 8080, 21, 443, 3306 using SO_REUSEADDR
- Step 3 – Threading: One dedicated thread spawned per port for parallel listening
- Step 4 – Accept: Each thread enters an infinite loop calling accept() on its socket
- Step 5 – Logging: On connection, IP, port, service name and timestamp are captured
- Step 6 – Persistence: Entry is appended to logs/honeypot_log.json
- Step 7 – Response: Fake 'ACCESS DENIED' banner is sent to the attacker, connection closed
- Step 8 – Visualization: Flask dashboard reads log file every 5 seconds and refreshes UI

2.3. Process Flow

Data flows through the system in the following direction:

- Attacker → TCP connection to honeypot port
- Socket listener → accepts connection, hands off to handler thread
- Connection handler → extracts addr[0] (IP) and datetime.now() (timestamp)
- Port map lookup → PORTS dict returns service name for the port number
- Entry dict → {ip, port, service, timestamp} saved to honeypot_log.json
- Fake banner → FAKE_BANNER bytes sent to attacker, socket closed
- Flask server → reads log file on /api/logs and /api/stats requests
- Browser dashboard → fetch() every 5 seconds, renders charts and tables

Step 1
Attacker
Initiates TCP connection to a honeypot port
Step 2
Socket binding
Sockets bound to ports 8080, 21, 443, 3306 using SO_REUSEADDR
Step 3
Threading engine
One dedicated daemon thread spawned per port for parallel listening
Step 4
Accept loop
Each thread calls accept() — extracts IP, port, service name & timestamp
Step 5
JSON logger
Entry {ip, port, service, timestamp} appended to honeypot_log.json
JSON log file
Step 6
Fake banner engine
ACCESS DENIED banner sent to attacker — connection closed immediately
Step 7
Flask dashboard
Reads honeypot_log.json every 5 seconds — serves /api/logs & /api/stats
Step 8
Admin dashboard
Real-time charts, attack log table, top IPs, port statistics
Legend — click any node to learn more
External actor
Network layer
Storage layer
Response layer
Dashboard layer
Persistent storage

2.4. Information Flow

The system is composed of the following major components:

Component	Description
Port Listener	Binds socket to each configured port, calls listen(5), and accepts connections continuously
Threading Engine	Spawns one daemon thread per port so all ports listen simultaneously without blocking
Connection Handler	Runs per-connection — extracts attacker IP and timestamp, identifies service
JSON Logger	Reads existing log file, appends new entry, writes back to disk with indent=2
Fake Response Engine	Sends multi-line fake EdTech student portal banner bytes to attacker and closes the socket
Flask Dashboard	Web server with /api/logs, /api/stats and / route rendering index.html
Stats Aggregator	Counts hits per IP, per port, per hour for the dashboard statistics panels
Chart Renderer	Client-side Chart.js line chart (timeline) and doughnut chart (port breakdown)

2.5. Key Design Considerations

- All port listeners run as daemon threads so they shut down cleanly when the main process exits
- SO_REUSEADDR is set on every socket to allow instant restart without 'Address already in use' errors
- Fake banner is identical across all ports to prevent attackers from fingerprinting the service
- Log file is structured as a JSON array so it can be consumed directly by any tool or script
- Dashboard auto-refreshes every 5 seconds without requiring a page reload
- Honeypot and dashboard are decoupled — only connected via the shared log file

2.6. API Catalogue

The system exposes two internal REST endpoints through the Flask dashboard. These are consumed only by the frontend JavaScript and are not meant for external integration.

Endpoint	Method	Response	Description
/	GET	HTML	Renders the main dashboard page
/api/logs	GET	JSON	Returns the full array of connection log entries
/api/stats	GET	JSON	Returns aggregated stats: hits, top IPs, top ports, time buckets

3. Data Design

The following internal function interfaces are defined:

Function	Input	Output	Description
init_log()	None	None	Creates log file if not present
save_log(entry)	Dict	None	Appends entry to JSON log
handle_connection(conn, addr, port)	Socket, Tuple, Int	None	Logs connection and sends banner
start_listener(port)	Int	None	Binds port and runs accept loop
read_logs()	None	List	Reads all log entries from disk

api_stats()	None	JSON	Returns aggregated dashboard stats

3.1. Data Model

The primary data model used by the system is a JSON array of log entries. Each entry is a flat dictionary with the following schema:

Field	Data Type	Description
ip	String	Attacker IP address extracted from socket addr
port	Integer	Port number attacker tried to connect to
service	String	Human-readable service name (e.g. Student Portal)
timestamp	String (YYYY-MM-DD HH:MM:SS)	Exact time the connection was received

3.2. Data Access Mechanism

- Log File: logs/honeypot_log.json — plain JSON array read and written on each connection
- Append Strategy: full file is loaded, new entry appended, full file rewritten — safe for project-scale loads
- Dashboard Reads: Flask endpoints open the log file, parse JSON, and return the result
- Polling: Browser fetches /api/stats and /api/logs every 5 seconds

3.3. Data Retention Policies

- Log File: retained indefinitely on disk — never auto-deleted during operation
- In-Memory State: no in-memory cache; log is always read fresh from disk
- Dashboard Display: shows last 100 records in the live connection log table for readability
- Full Data: complete log file remains available for export and offline analysis

3.4. Data Processing (ETL)

The system does not involve database migration — the log store is a flat JSON file. If the system is moved to a new host, copying the honeypot_log.json file is sufficient to preserve the full attack history. If required, the same data can be imported into PostgreSQL, MongoDB or an ELK stack for production deployments without modifying the honeypot core.

4. Interfaces

The system provides a single web-based interface through the Flask dashboard, accessible at http://127.0.0.1:5000. The interface consists of the following elements:

UI Element	Type	Description
Status Badge	HTML div	Shows MONITORING ACTIVE with a pulsing dot
Stat Cards (x4)	HTML cards	Total Hits, Unique IPs, Active Ports, Top Port
Attack Timeline	Chart.js line	Hits per time bucket over capture period
Top Attacker IPs	HTML list	Ranked list of source IPs with hit counts
Hits by Port	Chart.js doughnut	Proportional breakdown of attacks across ports
Port Services	HTML list	Per-port service label and hit count
Live Connection Log	HTML table	Timestamp, IP, port, service, status per event

5. State and Session Management

The honeypot system does not maintain user sessions as it has no user login. State is managed through the following components:

- Listener State: each daemon thread holds its own socket object and accept loop
- Connection State: every incoming connection is handled independently — no session carry-over
- Log State: persistent across restarts because it lives on disk in honeypot_log.json
- Dashboard State: stateless HTTP — every request re-reads the log file from scratch

6. Caching

The system does not use an in-memory cache for log data. Every /api/logs and /api/stats request reads the log file fresh from disk. This keeps the implementation simple and guarantees the dashboard always reflects the latest recorded events. Browser-side caching is also disabled through the 5-second polling interval. For production deployments at scale, a Redis layer could be added between the log file and Flask to avoid repeated disk reads — this is intentionally out of scope for the academic version.

7. Non-Functional Requirements

7.1. Security Aspects

- Fake banner hides all real server information from attackers
- Log file stored locally, not exposed to any external interface
- Dashboard accessible only on localhost during demo deployment
- All ports return identical fake responses to prevent service fingerprinting
- No user authentication required — the system is read-only monitoring

7.2. Performance Aspects

- Multi-threaded listeners allow concurrent connections on all 4 ports
- Daemon threads ensure clean shutdown without zombie processes
- JSON append-write pattern suitable for low to medium attack volumes
- Dashboard polls every 5 seconds — adjustable based on need
- SO_REUSEADDR enables instant socket rebinding after restart

7.3. Limitations

- Ports below 1024 (21, 443) may require admin/root privileges on some systems
- Local demo deployment only — no public IP in academic version
- JSON file storage not suitable for high-volume production traffic
- No geolocation of attacker IPs in this version
- No alerting — purely passive logging and visualization

8. References

- Python Socket Library Documentation — https://docs.python.org/3/library/socket.html
- Python Threading Documentation — https://docs.python.org/3/library/threading.html
- Flask Official Documentation — https://flask.palletsprojects.com
- Chart.js Documentation — https://www.chartjs.org/docs
- OWASP Honeypot Project — https://owasp.org/www-project-honeypot
- Cowrie Honeypot Reference — https://github.com/cowrie/cowrie
- T-Pot Multi-Honeypot Platform — https://github.com/telekom-security/tpotce
- MITRE ATT&CK Framework — https://attack.mitre.org